

北京理工大学C语言期末模拟试题含参考答案

一、单项选择题 - 试答 1

Question 1

分数: 2

已知结构定义如下:

```
struct sk
{
    int a;
    float b;
} data, *p;
```

如果 $p = \&data$; 则对于结构变量 $data$ 的成员 a 的正确引用是。

选择一个答案

☐ A. $p \rightarrow data.a$

☐ B. $p.data.a$

☒ C. $(*p).a$

☐ D. $(*)data.a$

Question 2

分数: 2

说明一个变量 s 来保存 20 以内整数的阶乘, 则该变量应说明为。

选择一个答案

☒ A. float 型

☐ B. long 型

☐ C. unsigned 型

☐ D. int 型

Question 3

分数: 2

下列四组标识符中, 每组有用逗号分开的 4 个标识符, 全都符合标准 C 标识符规定的一组是_____。

选择一个答案

- ☐ A. *4, trin, mA, Ma
- ☐ B. r_, _r, i, st'
- ☒ C. name, t3, x_y, _n3y
- ☐ D. function1, g, c10, k.c7

Question 4

分数: 2

已知 `int n = 1, m = 0`; 执行下述语句后, 变量 `m` 的值是。

```
switch( n )
{
    case 1:
        case 2: m = 1;
            break;
        case 3: m = 3;
        default: m = 2;
}
```

选择一个答案

- ☐ A. 3
- ☐ B. 0
- ☐ C. 2
- ☒ D. 1

Question 5

分数: 2

已知: `char str[ ][4] = { "abc", "def", "ghi" }, c, *p=&c`; 如果要使则执行“`printf( "%c", *p )`”语句后的输出结果为字符‘c’, 则赋值语句是。

选择一个答案

- ☐ A. `p = str[0][2];`
- ☒ B. `*p = str[0][2];`
- ☐ C. `p = str[1][3];`
- ☐ D. `*p = str[1][3];`

Question 6

分数: 2

已知: `int k = 2, m = 3, n = 4`; 则表达式 “ `!k && m || n` ” 的值是。

选择一个答案

☒ A. 1

☐ B. 3

☐ C. 2

☐ D. 0

Question 7

分数: 2

在下面的说明语句中为字符数组 **str** 赋初值，而 **str** 不能作为字符串使用的是。

选择一个答案

☐ A. `char str[8] = "Beijing";`

☒ B. `char str[7] = "Beijing";`

☐ C. `char str[10] = "Beijing";`

☐ D. `char *str = "Beijing";`

Question 8

分数: 2

已知在主调函数中有说明语句如下：

```
int n, data[10][6];  
double sum, *q;
```

函数的调用语句是： `q = func(&n, data );`

则函数 **func** 的原型是 _____ 。

选择一个答案

☐ A. `double func( int *x, int **data );`

☐ B. `double func( int x, int data[][] );`

☐ C. `double *func( int *x, int **data );`

☒ D. `double *func( int *x, int data[][6] );`

Question 9

分数: 2

已知 `inti`, `k = 4`; 则执行循环语句:

```
for( i =0; ++i< k; i++ )  
    printf("%d\n", i);
```

最后一行输出的数值是。

选择一个答案

☐ A. 4

☒ B. 3

☐ C. 5

☐ D. 2

保存但不提交

提交所有答案并结束

二、程序填空

分数: 2

以下函数使用合并排序法, 将两个已经按照从小到大的排序的整型数组, 合并到新的数组, 新数组仍按照从小到大的顺序排序, 数值相同的数据在新数组中只保存一次。参数 `a`、`b` 是两个排好序的数组的首地址, 参数 `c` 是合并后数组的首地址, 参数 `m`、`n` 分别是数组 `a`、`b` 的长度, 函数的返回值是数组 `c` 的长度。

```
combine(int *a, int m, int *b, int n, int *c )  
{  
    inti = 0, j = 0, k = 0;  
    while( 【1】 )  
        c[k++] = a[i] < b[j] ? a[i++] : b[j++];  
    while( i < m )  
        c[k++] = a[i++];  
    while( j < n )  
        c[k++] = b[j++];  
    for( i = 0; i < k-1; i++ )  
    {  
        while( c[i+1] == c[i] )  
        {  
            for( j = 【2】 ; j < k-1; j++ )  
                c[j] = c[j+1];  
            【3】 ;  
        }  
    }  
}
```

```
        return( k );  
    }
```

【1】:

选择一个答案

- ☐ A. $i \leq m \parallel j \leq n$
- ☐ B. $i < m \parallel j < n$
- ☐ C. $i \leq m \&\& j \leq n$
- ☐ D. $i < m \&\& j < n$

Question 2

分数: 2

【2】:

选择一个答案

- ☐ A. 1
- ☐ B. i
- ☐ C. 0
- ☐ D. $i + 1$

Question 3

分数: 2

【3】:

选择一个答案

- ☐ A. $i--$;
- ☐ B. $i++$;
- ☐ C. $k--$;
- ☐ D. $k++$;

Question 4

分数: 2

从键盘输入 3 个学生的数据，将它们存入当前文件夹下名为"student"的文件中。

```
#include <stdio.h>
```

```
#define SIZE 3
```

```

struct student
{
    long num;
    char name[10];
    int age;
    char address[10];
}stu[SIZE], out;

main ( )
{
    FILE *fp;
    int i;
    for( i = 0; i < SIZE; i++ )
    {
        printf( "Input student %d:", i + 1 );
        scanf( "%ld%s%d%s", &stu[i].num, stu[i].name, &stu[i].age, stu[i].address );
    }
    if( ( fp = fopen( "student", 【4】 ) ) == NULL )
    {
        printf( "Cannot open file.\n" );
        exit(1);
    }
    for( i = 0; i < SIZE; i++ )
        if( fwrite( 【5】 , sizeof(struct student), 1, fp ) 【6】 1 )
            printf("File write error.\n");
    fclose(fp);
}

```

【4】：

选择一个答案

- ☐ A. "a"
- ☐ B. "w"
- ☐ C. "ab"
- ☐ D. "wb"

Question 5

分数: 2

【5】：

选择一个答案

- ☐ A. stu[i]
- ☐ B. *stu[i]

☐ C. stu

☐ D. &stu[i]

Question 6

分数: 2

【6】:

选择一个答案

☐ A. == 1

☐ B. == NULL

☐ C. != NULL

☐ D. != 1

• [查看](#)

• [提交](#)

• [结果](#)

• [提交历史](#)

• [相似度](#)

三、编程题_1

成绩: 10 / 折扣: 1

输入行数 n 值和首字母，输出由大写英文字母围起的空心三角形。

无论输入的首字母是大写或小写，输出的字母均是大写，且字母输出是循环的，即输出字母 Z 后再输出的是字母 A。

例：输入：5 m✓

屏幕输出：

```
      M✓
     N  N✓
    O    O✓
   P      P✓
  Q R S T U T S R Q✓
```

测试输入 		期待的输出 	时间限制 	内存限制 	额外进程 
以文本方式显示			1 秒	64M	0
以文本方式显示					
测试用例	1.	M*			
	2.	N N*			
1	1. 5 m*	3. O O*			
		4. P P*			
		5. QRSTUTSRQ*			
开启时间	2011 年 12 月 30 日星期五 09:00				
折扣时间	2012 年 01 月 6 日星期五 00:00				
关闭时间	2012 年 01 月 6 日星期五 00:00				

允许迟交: 否

- [查看](#)
- [提交](#)
- [结果](#)
- [提交历史](#)
- [相似度](#)

三、编程题_2

成绩: 10 / 折扣: 1

现有 n 元整钱，到银行兑换成 1 角、2 角、5 角的硬币。编写程序，输入整钱数 n ，输出可能兑换的各种硬币数量的组合。

要求编写程序，当输入整钱数 N 时，输出每种兑换方案的硬币枚数。如果有多种兑换方案，按 1 角硬币数量的升序输出。

例如，整钱数 2 元，

输入: 2

输出:

```
num: $0.1 $0.2 $0.5
1: 1 2 3
2: 1 7 1
3: 2 4 2
4: 3 1 3
```



```
5: 3 6 1
6: 4 3 2
7: 5 5 1
8: 6 2 2
9: 7 4 1
10: 8 1 2
11: 9 3 1
12: 11 2 1
13: 13 1 1
```

其中第一行是表头。从第二行起每行为一种兑换方案；第一列为方案编号，第二列为 1 角硬币数量，第三列为 2 角硬币数量，第四列为 5 角硬币数量。

编写求兑换方案的函数：int findm(int n, int mn[][3])

其中：n 是整钱数，mn 是保存兑换方案的数组名；函数的返回值是兑换方案个数。要求：输出的兑换方案按照 1 角硬币数量的升序排列，并且每种硬币的数量必须大于 0。

注意：只提交自编的函数 findm，不要提交主函数。给定的程序已经控制好了输出的格式，你只要将你的方案按要求放入数组 mn 中即可。

预设代码

前置代码

view plaincopy to clipboardprint?

```
1. /* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
2.
3. #include <stdio.h>
4.
5. int findm( int , int [][][3] );
6.
7. int main()
8. { int i, n, num, combm[500][3];
9.
10.     scanf("%d",&n);
11.     num = findm( n, combm );
12.     printf("num: $0.1 $0.2 $0.5\n" );
13.     for ( i=0; i < num; i++ )
14.         printf("%3d:%5d%5d%5d\n", i+1, combm[i][0], combm[i][1], co
            mbm[i][2] );
15.     return 0;
16. }
17.
18. /* int findm( int n, int mn[][3] ) */
```

```

19./* { ..... */
20./* } */
21.
22./* PRESET CODE END - NEVER TOUCH CODE ABOVE */

```

```

/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
#include <stdio.h>

int findm( int , int [][][3] );

int main()
{ int i, n, num, combm[500][3];

  scanf("%d",&n);
  num = findm( n, combm );
  printf("num: $0.1 $0.2 $0.5\n");
  for ( i=0; i < num; i++)
    printf("%3d:%5d%5d%5d\n", i+1, combm[i][0], combm[i][1], combm[i][2]);
  return 0;
}

/* int findm( int n, int mn[][][3] ) */
/* { ..... */
/* } */

/* PRESET CODE END - NEVER TOUCH CODE ABOVE */

```

测试输入?	期待的输出?	时间限制?	内存限制?	额外进程?
以文本方式显示		1 秒	64M	0
测试用例	1. num: \$0.1 \$0.2 \$0.5* 2. 1: 1 2 3* 3. 2: 1 7 1* 4. 3: 2 4 2* 5. 4: 3 1 3* 6. 5: 3 6 1* 7. 6: 4 3 2* 8. 7: 5 5 1* 9. 8: 6 2 2* 10. 9: 7 4 1* 11. 10: 8 1 2* 12. 11: 9 3 1* 13. 12: 11 2 1* 14. 13: 13 1 1*			

开启时间 2011 年 12 月 30 日星期五 09:00
 折扣时间 2012 年 01 月 6 日星期五 00:00
 关闭时间 2012 年 01 月 6 日星期五 00:00

允许迟交: 否

• [查看](#)

- 提交
- 结果
- 提交历史
- 相似度

三、编程题_3

成绩: 10 / 折扣: 1

编写一个函数，使用递归算法求下述函数的值。

$$f(n,x) = \begin{cases} 1 & \text{当 } n == 0 \text{ 时} \\ x & \text{当 } n == 1 \text{ 时} \\ 2 * f(n-1,x) + 3 * f(n-2,x) & \text{当 } n > 1 \text{ 时} \\ -1 & \text{其它} \end{cases}$$

函数原型如下：

```
double findf(int n, double x)
```

参数说明：n 项数($n > 4$), x 自变量；返回值是函数值。

例如输入：5 4.0，输出：304.00

注意：仅提交自编的 `findf` 函数，不提交 `main` 函数。

预设代码

a3_3.c

view plaincopy to clipboardprint?

```
1. /* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
2.
3. #include <stdio.h>
4. int main()
5. { int n;
6.   double f, s, findf( );
7.   scanf("%d%lf",&n,&f);
8.   s = findf(n, f);
9.   printf("%.2lf\n", s);
10.  return 0;
11. }
12.
13. /* PRESET CODE END - NEVER TOUCH CODE ABOVE */
```

```
/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */

#include <stdio.h>
int main()
{
    int n;
    double f, s, findf( );
    scanf("%d%lf", &n, &f);
    s = findf(n, f);
    printf("%.2lf\n", s);
    return 0;
}

/* PRESET CODE END - NEVER TOUCH CODE ABOVE */
```

测试输入 ?		期待的输出 ?	时间限制 ?	内存限制 ?	额外进程 ?
以文本方式显示		以文本方式显示			
测试用例			1 秒	64M	0
1	1. 5 4.0*	1. 304.00*			

开启时间	2011 年 12 月 30 日星期五 09:00
折扣时间	2012 年 01 月 6 日星期五 00:00
关闭时间	2012 年 01 月 6 日星期五 00:00

允许迟交: 否

- [查看](#)
- [提交](#)
- [结果](#)
- [提交历史](#)
- [相似度](#)

三、编程题_4

成绩: 10 / 折扣: 1

功能要求：在带表头的单向链表中，找到其中的最小值结点，删除该结点之后的所有结点。

说明：(1) 用带表头的单向链表的方式保存输入的各整数数值，每一个结点的数值域保存一个数值。

(2) 若具有最小数值的结点有多个，则选取其中位于链表最前端的一个结点。预设的代码包括主函数、建立链表函数、输出链表函数，请编写找到链表中最小值结点，并将其之后所有结点删除的函数。

结构的定义：

```
struct node
```

```
{ int num;
```

```
struct node *next; }
```

```
typedef struct node NODE;
```

```
typedef struct node * PNODE;
```

函数的原型：void delAftMin(PNODE head)，其中：参数 head 是单向链表的头指针。

预设代码

前置代码

[view plain](#)[copy](#) [to clipboard](#)[print?](#)

```
1. /* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
2.
3. #include <stdio.h>
4. #include <stdlib.h>
5.
6. struct node
7. {   int data;
8.     struct node * next;
9. } ;
10.
11. typedef struct node NODE;
12. typedef struct node * PNODE;
13.
14. PNODE constructlist( PNODE head, int num );
15. void outlist( PNODE head );
16. void delAftMin( PNODE head );
17.
18. int main ( )
19. {   int num=1;
20.     PNODE head;
21.
22.     head = (PNODE)malloc( sizeof(NODE) );
```

```

23.     head->next = NULL;
24.     head->data = -1;
25.
26.     while ( num!=0 )
27.     {     scanf("%d", &num);
28.         if ( num!=0 )
29.             constructlist (head, num);
30.     }
31.     delAftMin( head );
32.     outlist( head );
33.     return 0;
34.}
35.
36.PNODE constructlist( PNODE head, int num )
37.{     PNODE p;
38.     p = (PNODE)malloc( sizeof(NODE) );
39.     p->data = num;
40.     p->next = head->next;
41.     head->next = p;
42.     return head;
43.}
44.
45.void outlist( PNODE head )
46.{     PNODE p;
47.     p = head->next;
48.     while ( p != NULL )
49.     {     printf("%d\n", p->data);
50.         p = p->next;
51.     }
52.}
53.
54./* This is an example for list. Please programme your code like it
   .
55.void delAftMin( PNODE head )
56.{
57.}
58.*/
59.
60./* PRESET CODE END - NEVER TOUCH CODE ABOVE */

```



```

/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node * next;
};

typedef struct node NODE;
typedef struct node * PNODE;

PNODE constructlist( PNODE head, int num );
void outlist( PNODE head );
void delAftMin( PNODE head );

int main ( )
{
    int num=1;
    PNODE head;

    head = (PNODE)malloc( sizeof(NODE) );
    head->next = NULL;
    head->data = -1;
}

```

测试输入 		期待的输出 	时间限制 	内存限制 	额外进程 
以文本方式显示		以文本方式显示	无限制	64M	0
测试用例					
1	1. 30 40 20 10 0*	1. 10*			
以文本方式显示		以文本方式显示	无限制	64M	0
测试用例					
3	1. 40 20 40 30 0*	1. 30* 2. 40* 3. 20*			
开启时间	2011 年 12 月 30 日星期五 09:00				
折扣时间	2012 年 01 月 6 日星期五 00:00				
关闭时间	2012 年 01 月 6 日星期五 00:00				

允许迟交: 否